

$\|$, $\&\&$, $!$

Boolean operators

Control Flow

We will translate control flow statements such as if-else statements, while statements. Translation of control flow statements are tied with Boolean expressions. For writing Boolean expression you will need boolean operators.

Boolean operator are applied to:

- a) Boolean variables
- b) Relational expression with relational operator

In programming languages, boolean expressions are often used to:

Alter the flow of control: Boolean expressions are used as conditional expressions in statements that alter the flow of control. For example, in *if (E) S*, E must be true to reach S

Compute logical values: A boolean expression can represent true or false as values.

$>$, $<$, $>=$, $<=$, $=$, $!=$

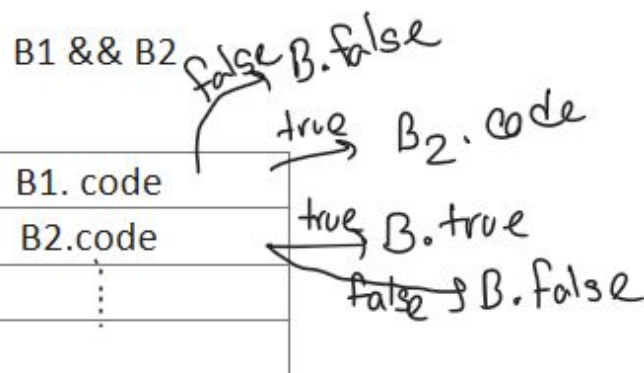
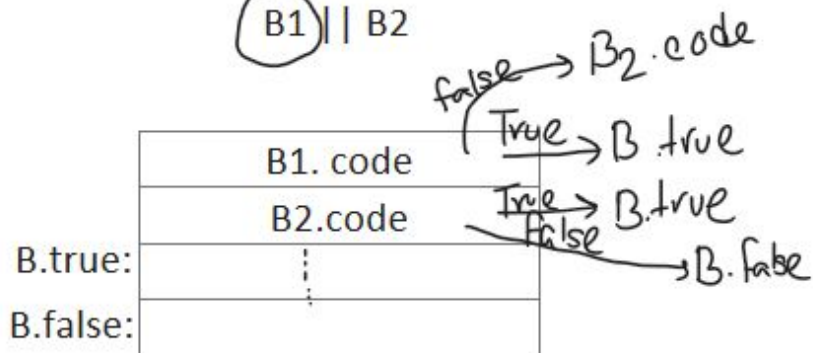
relational operators

SDD for Generating Three Address Code for Booleans

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = \text{newlabel}()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel \text{label}(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = \text{newlabel}()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel \text{label}(B_1.true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel \text{gen('it' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$ $\parallel \text{gen('goto' } B.false)$
$B \rightarrow \text{true}$	$B.code = \text{gen('goto' } B.true)$
$B \rightarrow \text{false}$	$B.code = \text{gen('goto' } B.false)$

$I \&\& I \rightarrow \text{true}$
 $F \&\& T \rightarrow \text{false}$
 $\textcircled{T} \&\& \textcircled{F} \rightarrow \text{false}$
 $F \&\& F \rightarrow \text{false}$

$\textcircled{B1} \parallel B2$



B.code : contains codes of B

B.true : label to which control flows if B is true

B.false : label to which control flows if B is false

SDD for Generating Three Address Code for Flow-of-Control Statements

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if} (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if} (B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while} (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

$S.code$: contains codes of S

$S.next$: label of a instruction immediately after S

Input String: while (a + b > 2 || a != 3)
 a = b + 3;
 y = 3;

P → S { S.next = newlabel();
 P.code = S.code || label(S.next); }

S → if (B) S₁ { B.true = newlabel();
 B.false = S₁.next = S.next;
 S.code = B.code || label(B.true) || S₁.code; }

if (B) S₁ else S₂ { B.true = newlabel();
 B.false = newlabel();
 S₁.next = S₂.next = S.next;
 S.code = B.code || label(B.true) ||
 S₁.code || gen('goto' S.next)
 || label(B.false) || S₂.code; }

while (B) S₁ { begin = newlabel(); B.true = newlabel();
 B.false = S.next; S₁.next = begin;
 S.code = label(begin) || B.code ||
 label(B.true) || S₂.code ||
 gen('goto' begin); }

id = E; { S.code = E.code ||
 gen('top.get(id.lexeme)' '=' E.addr); }

S → S₁ S₂ { S₁.next = newlabel(); S₂.next = S.next;
 S.code = S₁.code || label(S₁.next) || S₂.code; }

B → B₁ || B₂ { B₁.true = B.true; B₁.false = newlabel();
 B₂.true = B.true;
 B₂.false = B.false;
 B.code = B₁.code ||
 label(B₁.false) || B₂.code; }

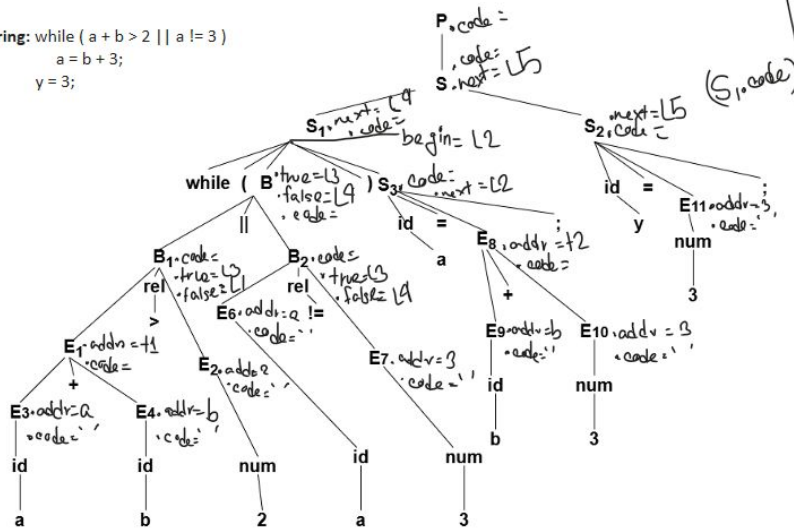
! B₁ { B₁.true = B.false; B₁.false = B.true;
 B.code = B₁.code; }

E₁ rel E₂ { B.code = E₁.code || E₂.code ||
 gen('if' E₁.addr rel.op E₂.addr 'goto' B.true)
 || gen('goto' B.false); }

E → E₁ + E₂ { E.addr = new Temp();
 E.code = E₁.code || E₂.code ||
 gen(E.addr = E₁.addr + E₂.addr); }

id { E.addr = top.get(id.lexeme); E.code = ''; }

num { E.addr = num.lexval; E.code = ''; }



L2;

t1 = a + b (E₁.code)
 if t1 > 2 goto L3
 goto L1
 L1:
 if a != 3 goto L3
 goto L4
 L3:
 t2 = b + 3 (E₆.code)
 a = t2
 goto L2
 L4:
 y = 3 (S₂.code)

(B₁.code) (B₂.code) (S₁.code) (S₂.code)

(P₁.code) (P₂.code) (P₃.code)

(P₄.code)

(P₅.code)

(P₆.code)

(P₇.code)

(P₈.code)

(P₉.code)